

Arbre Couvrant de Poids Minimum (Minimum Spanning Tree MST)

Masters

→ Arbres:

(Rappel)

Définition: un arbre est un graphe connexe sans cycle.

Proposition: Pour un graphe $G=(S,A)$, d'ordre n et de taille m , il y a équivalence entre:

1) $G=(S,A)$ est un arbre

2) G est connexe et $m=n-1$

3) G est sans cycle et $m=n-1$

4) G est connexe et la suppression d'une arête quelconque le déconnecte (non connexe)

5) G est sans cycle et l'ajout d'une arête quelconque crée un cycle.

6) entre deux sommets quelconques, il existe une et une seule chaîne (chaîne unique)

→ Arbre Couvrant de poids minimum. ($M \leq T$).

↳ Soit $G = (S, A)$ un graphe pondéré avec $w : A \rightarrow \mathbb{R}$.

Le poids d'un graphe est la somme des poids des arêtes

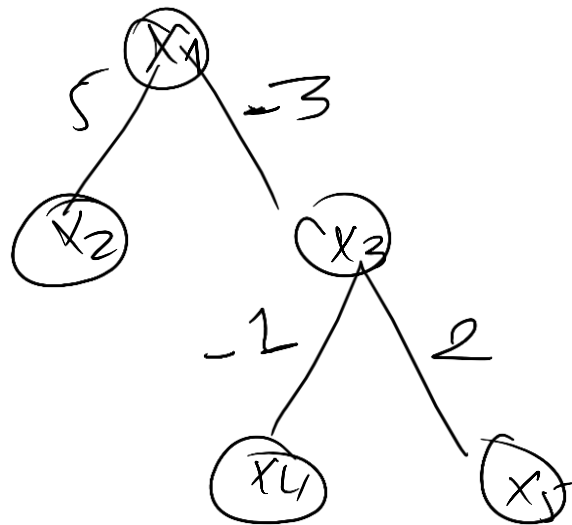
$$W(G) = \sum_{e \in A} w(e).$$

$\uparrow$ cont. de l'arête "e"

~~$\mathbb{R}$~~

arête (x_2, x_3)

Exemple:



$$w(x_1, x_3) = -3$$

$$\begin{aligned} W(G) &= \sum_{e \in A} w(e) \\ &= 5 - 3 - 1 + 2 \\ &= 3 \end{aligned}$$

→ On cherche un graphe partiel de G qui soit un arbre et qui soit de coût minimal.

$\min \{ w(T) \mid T \text{ est arbre et graphe partiel de } G \}$

↳ Remarques:

- le problème a une solution (on des solutions) si et seulement (ssi) le graphe initialement donné est **connexe**.

→ si tous les coûts sont positifs, le graphe partiel de coût minimal est forcément un arbre.

↳ Principe: (Algorithme de Kruskal) "Kruskal"

- Trier les arêtes par ordre des poids croissants

- Tant que l'on n'a pas retenu $(n-1)$ arêtes faire {

• considérer la ¹ère arête non examinée

• si elle forme un cycle avec les arêtes
précédemment retenues

• alors rejeter cette arête

• sinon conserver cette arête }

↳ Détection de cycles.

Nous allons gérer l'évolution des composantes connexes au fur et à mesure du choix d'arêtes.

→ Initialement, chaque sommet constitue une composante connexe (possédant un numéro de composante connexe cc_i)

→ Par chaque arête (u, v)

↳ si u et v ∈ à la même composante connexe ($cc(u) = cc(v)$) alors l'arête (u, v) est rejetée

↳ sinon, l'arête (u, v) est acceptée et on fusionne les composantes connexes.

→ une écriture de l'algorithme Kruskal.

Kruskal ($G = (S, A, w)$)

1 Trier les arêtes par poids $e_1, e_2, \dots, e_m$

Pour

Pour $v \in S$ faire {

D. **ajouter** (v)

$T \leftarrow \emptyset, k \leftarrow 0, j \leftarrow 1$

Tant que $k \leq n-1$ faire {

$(u, v) \leftarrow e_j$

Si D. **ensemble** (u) $\neq$ D. **ensemble** (v) alors {

$T \leftarrow T \cup \{e_j\}$

D. **union** (u, v)

$k \leftarrow k+1$

} $j \leftarrow j+1$ }

chaque sommet
reçoit un numéro
de composante connexe

fusionnement des
composantes connexes

→ Implémentation naïve des fonctions $\text{ajouter}(u)$, $\text{ensemble}(u)$ et $\text{union}(u, v)$:

↳ On maintient un tableau cc où $cc[u]$ est le numéro de la composante connexe du sommet u .

Ajouter u

{ $cc[u] \leftarrow i$
}

↳ numéro de la composante
connexe du sommet u

Ensemble u

{ Retourner $cc[u]$.

}

union(u, v)

{ temp $\leftarrow c(u)$

Pour chaque sommet $y \in S'$ faire {
si $c[y] = \text{temp}$ alors $c[y] \leftarrow c[v]$.
}

Exercice: Implémentation avec arbres orientés.

On maintient une forêt, chaque sommet étant un arbre. L'identifiant (Id) de l'ensemble est la racine de l'arbre.

Pour chaque sommet on garde

- Son parent

- Son rang (la hauteur du sous-arbre enraciné dans ce sommet.)

gestion: Proposer une écriture pour les fonctions Ajouter(u), Ensemble(u) et union(u, v).